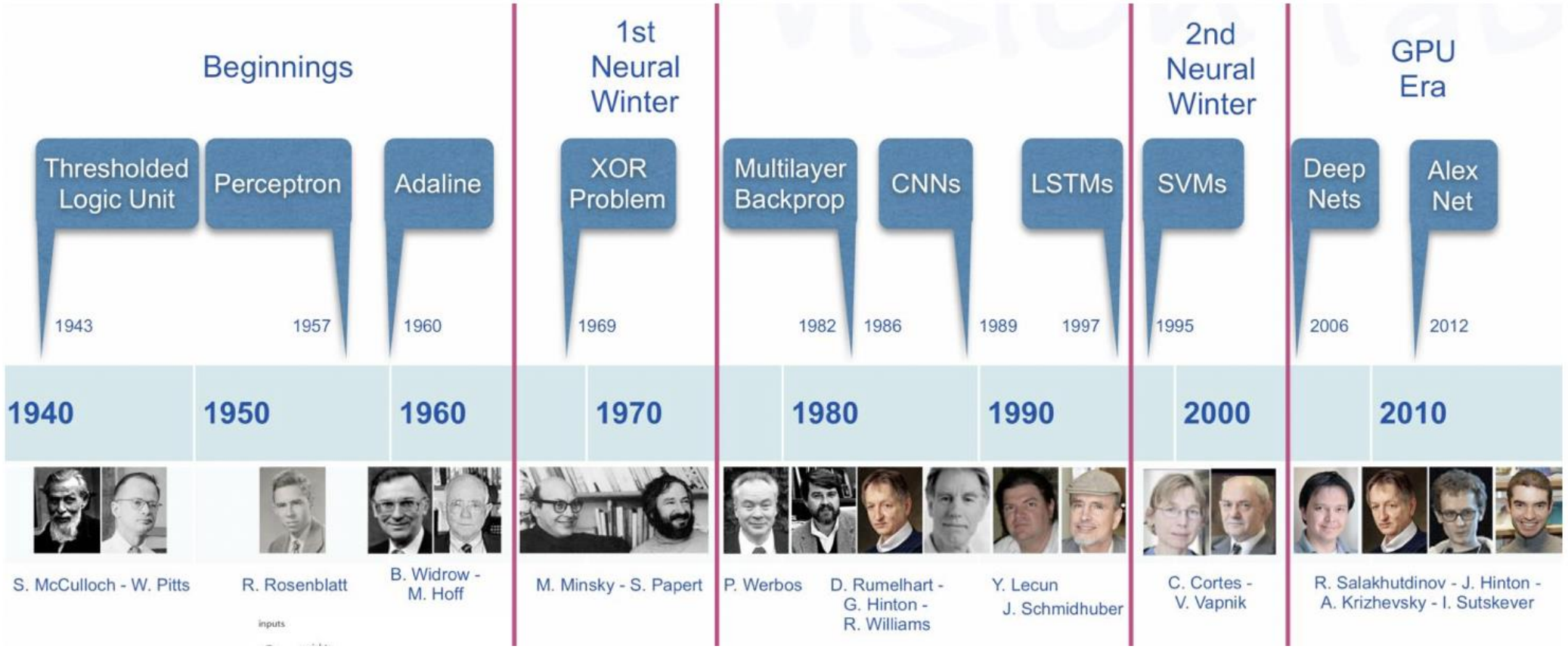
Support Vector Machines

Lê Anh Cường

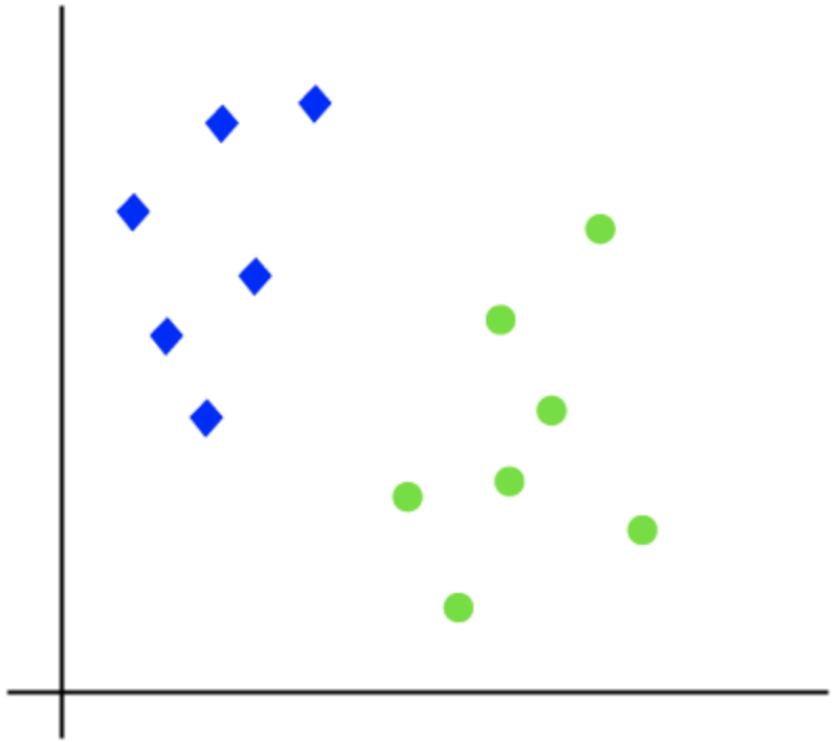
TDTU

History of Neural Network



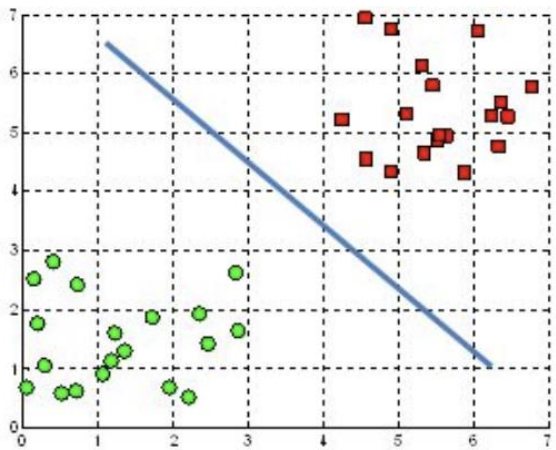
What is SVM

- Suppose we have a dataset that has two classes (green and blue). We want to classify that the new data point as either blue or green.

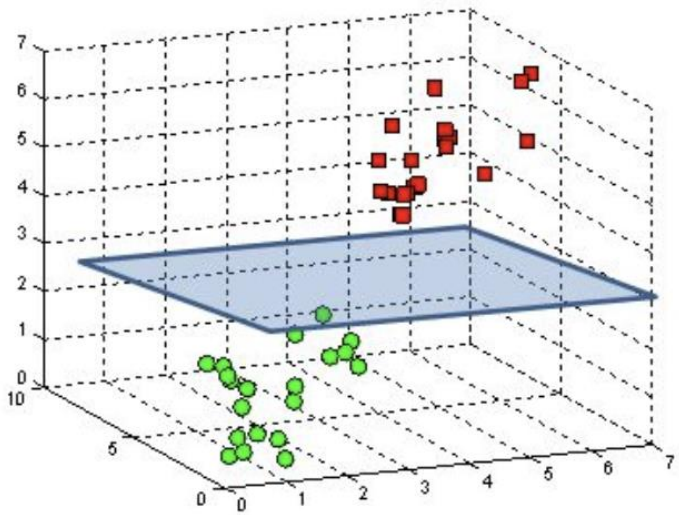


Hyperplane

A hyperplane in $\mathbb{R}^2$ is a line



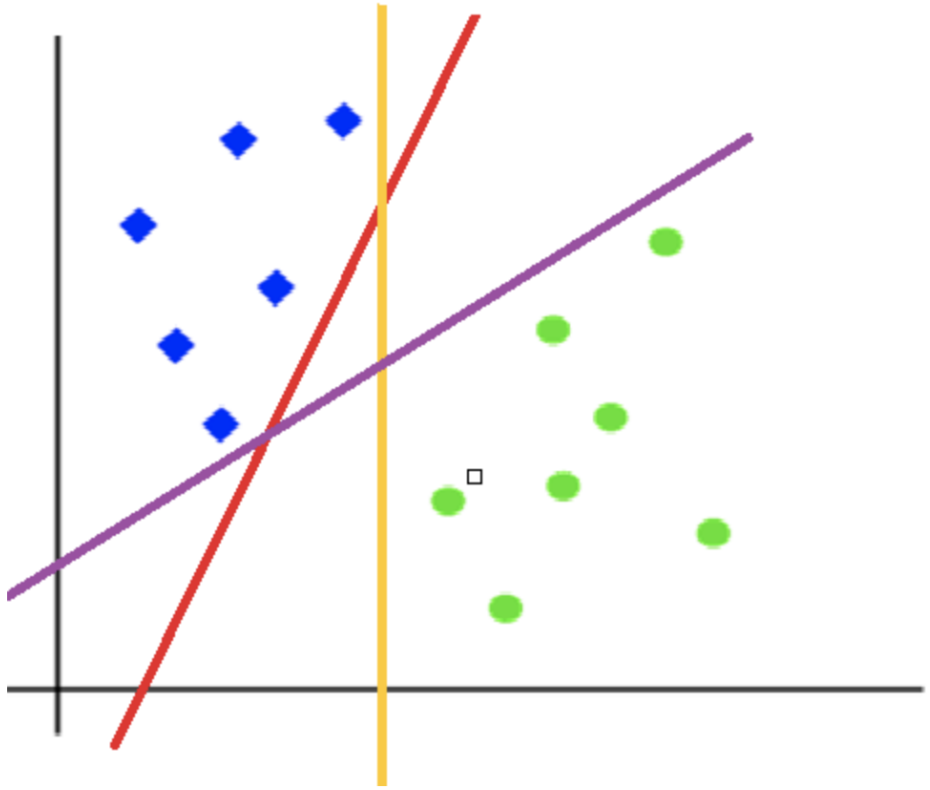
A hyperplane in $\mathbb{R}^3$ is a plane



What is SVM

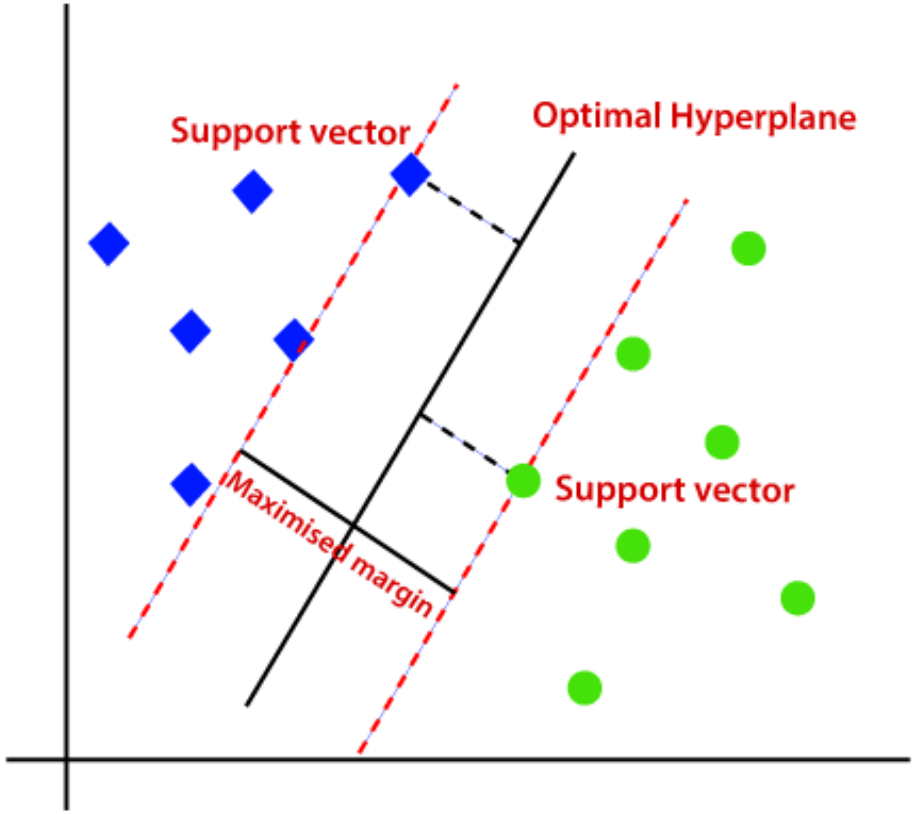
- To classify these points, we can have many decision boundaries, but the question is which is the best and how do we find it?

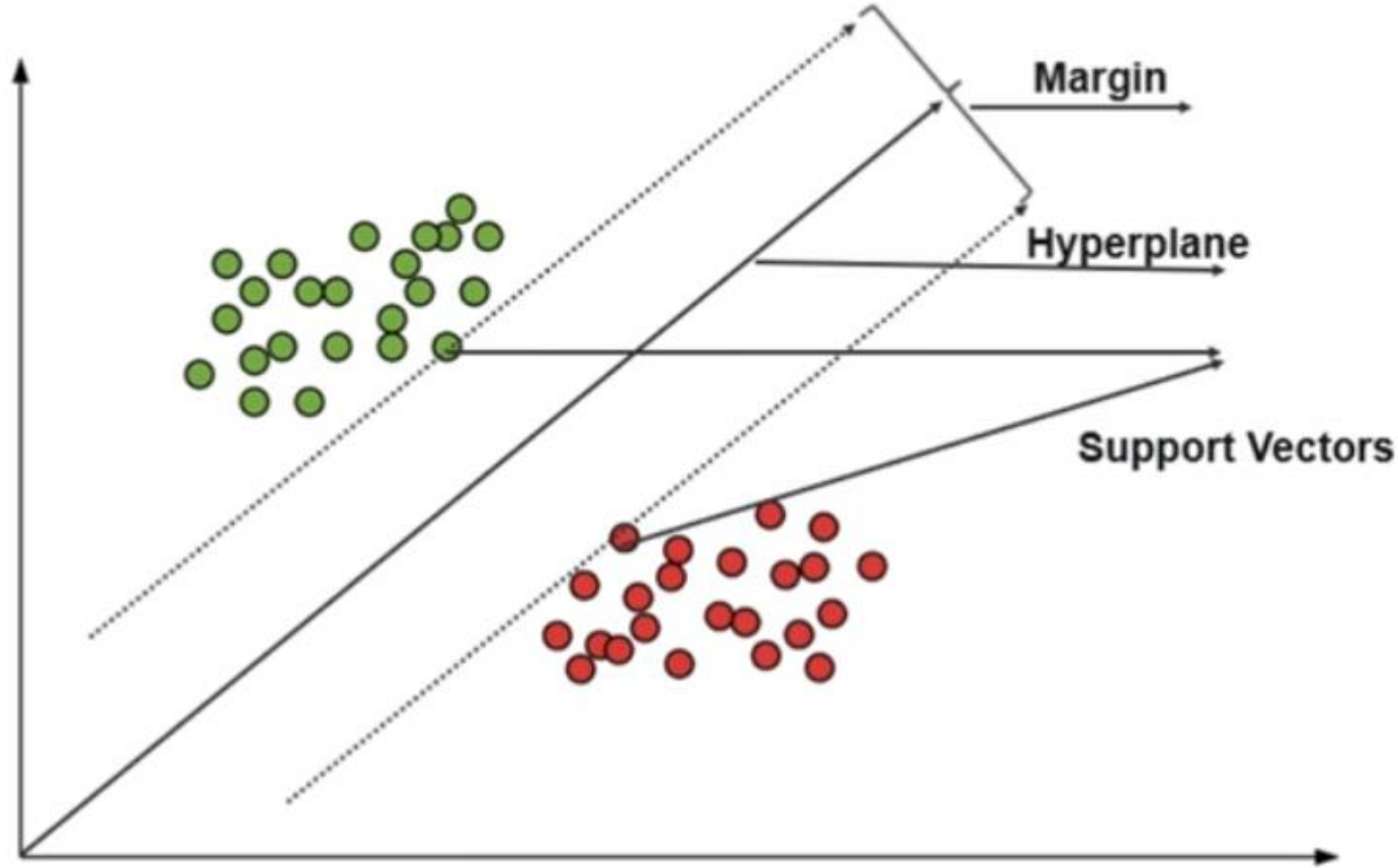
- Since we are plotting the data points in a 2-dimensional graph we call this decision boundary a **straight line** but if we have more dimensions, we call this decision boundary a **“hyperplane”**



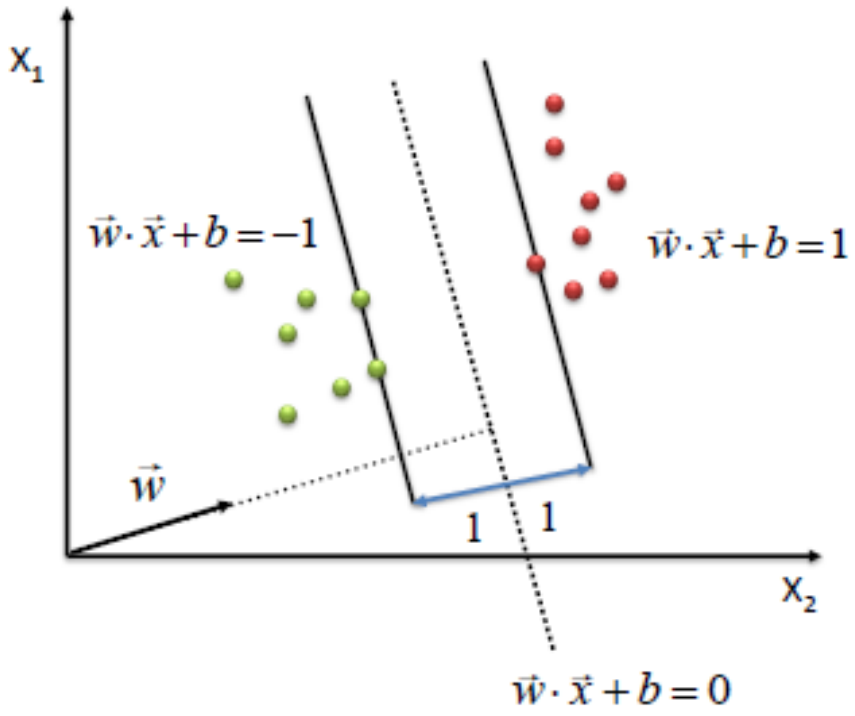
SVM aims to maximize the margin

- The best hyperplane is that plane that has the maximum distance from both the classes, and this is the main aim of SVM. This is done by finding different hyperplanes which classify the labels in the best way then it will choose the one which is farthest from the data points or the one which has a maximum margin.




Margin
Hyperplane
Support Vectors

SVM Learning: Maximize Margin

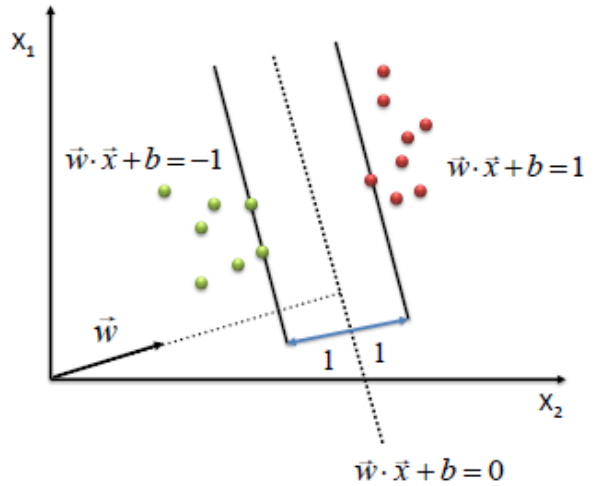


$$\max \frac{2}{\|w\|}$$

s.t.

$(w\cdot x+b)\geq 1, \forall x$ of class 1

$(w\cdot x+b)\leq -1, \forall x$ of class 2



$$\max \frac{2}{\|w\|}$$

s.t.

$(w\cdot x+b)\geq 1, \forall x$ of class 1

$(w\cdot x+b)\leq -1, \forall x$ of class 2

Goal: we want to find the hyperplane (i.e. decision boundary) linearly separating our classes. Our boundary will have equation: $\mathbf{w}^T\mathbf{x} + b = 0$.

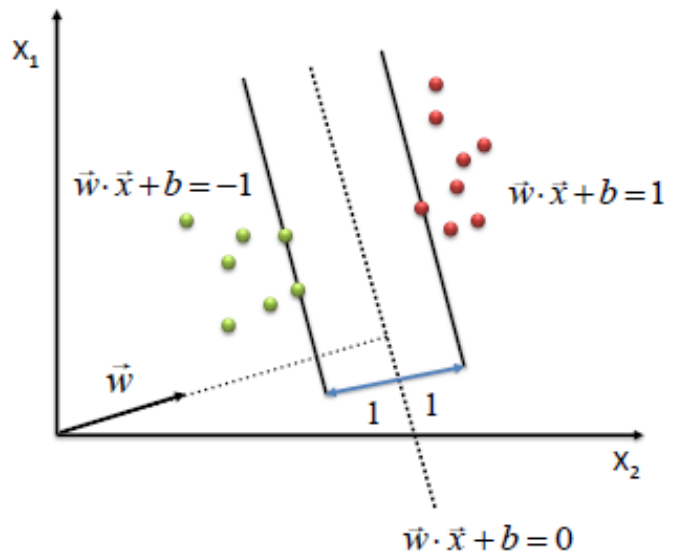
Anything above the decision boundary should have label 1.

i.e., $\mathbf{x_i}$ s.t. $\mathbf{w}^T\mathbf{x_i} + b > 0$ will have corresponding $y_i = 1$.

Similarly, anything below the decision boundary should have label -1.

i.e., $\mathbf{x_i}$ s.t. $\mathbf{w}^T\mathbf{x_i} + b < 0$ will have corresponding $y_i = -1$.

You'll notice that we will now have some space between our decision boundary and the nearest data points of either class. Thus, let's rescale the data such that anything on or above the boundary $\mathbf{w}^T\mathbf{x} + b = 1$ is of one class (with label 1), and anything on or below the boundary $\mathbf{w}^T\mathbf{x} + b = -1$ is of the other class (with label -1).



$$\max \frac{2}{\|w\|}$$

s.t.

$(w\cdot x+b)\geq 1, \forall x$ of class 1

$(w\cdot x+b)\leq -1, \forall x$ of class 2

First note that the two lines are parallel, and thus share their parameters $\mathbf{w}, b$. Pick an arbirary point $\mathbf{x_1}$ to lie on line $\mathbf{w}^T\mathbf{x} + b = -1$. Then, the closest point on line $\mathbf{w}^T\mathbf{x} + b = 1$ to $\mathbf{x_1}$ is the point $\mathbf{x_2} = \mathbf{x_1} + \lambda\mathbf{w}$ (since the closest point will always lie on the perpendicular; recall that the vector $\mathbf{w}$ is perpendicular to both lines). Using this formulation, $\lambda\mathbf{w}$ will be the line segment connecting $\mathbf{x_1}$ and $\mathbf{x_2}$, and thus, $\lambda\|\mathbf{w}\|$, the distance between $\mathbf{x_1}$ and $\mathbf{x_2}$, is the shortest distance between the two lines/boundaries. Solving for λ:

$$\Rightarrow \mathbf{w}^T\mathbf{x_2} + b = 1 \text{ where } \mathbf{x_2} = \mathbf{x_1} + \lambda\mathbf{w}$$

$$\Rightarrow \mathbf{w}^T(\mathbf{x_1} + \lambda\mathbf{w}) + b = 1$$

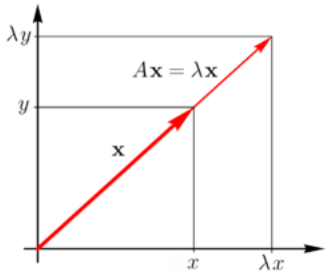
$$\Rightarrow \mathbf{w}^T\mathbf{x_1} + b + \lambda\mathbf{w}^T\mathbf{w} = 1 \text{ where } \mathbf{w}^T\mathbf{x_1} + b = -1$$

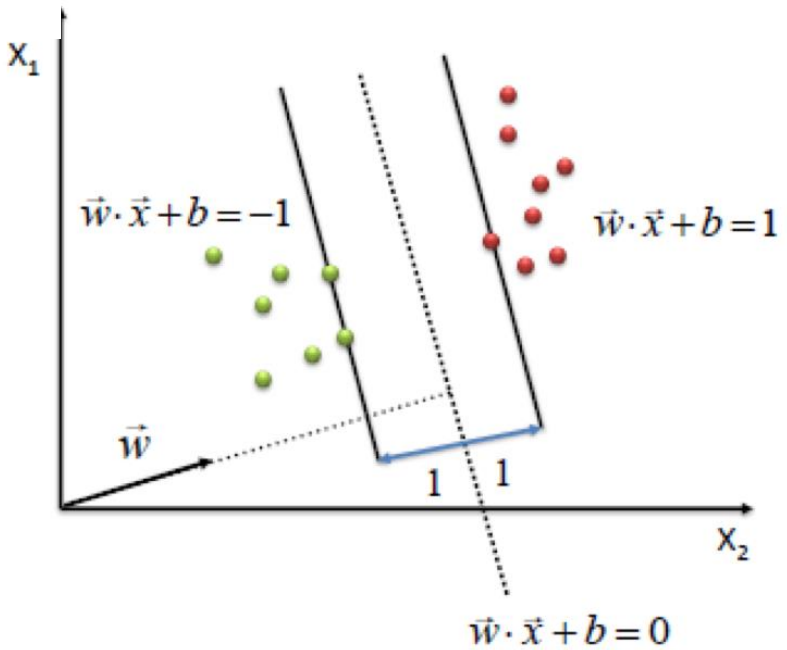
$$\Rightarrow -1 + \lambda\mathbf{w}^T\mathbf{w} = 1$$

$$\Rightarrow \lambda\mathbf{w}^T\mathbf{w} = 2$$

$$\Rightarrow \lambda = \frac{2}{\mathbf{w}^T\mathbf{w}} = \frac{2}{\|\mathbf{w}\|^2}$$

$$\max \frac{2}{\|w\|}$$





Hard margin

This *quadratic programming* problem is expressed as:

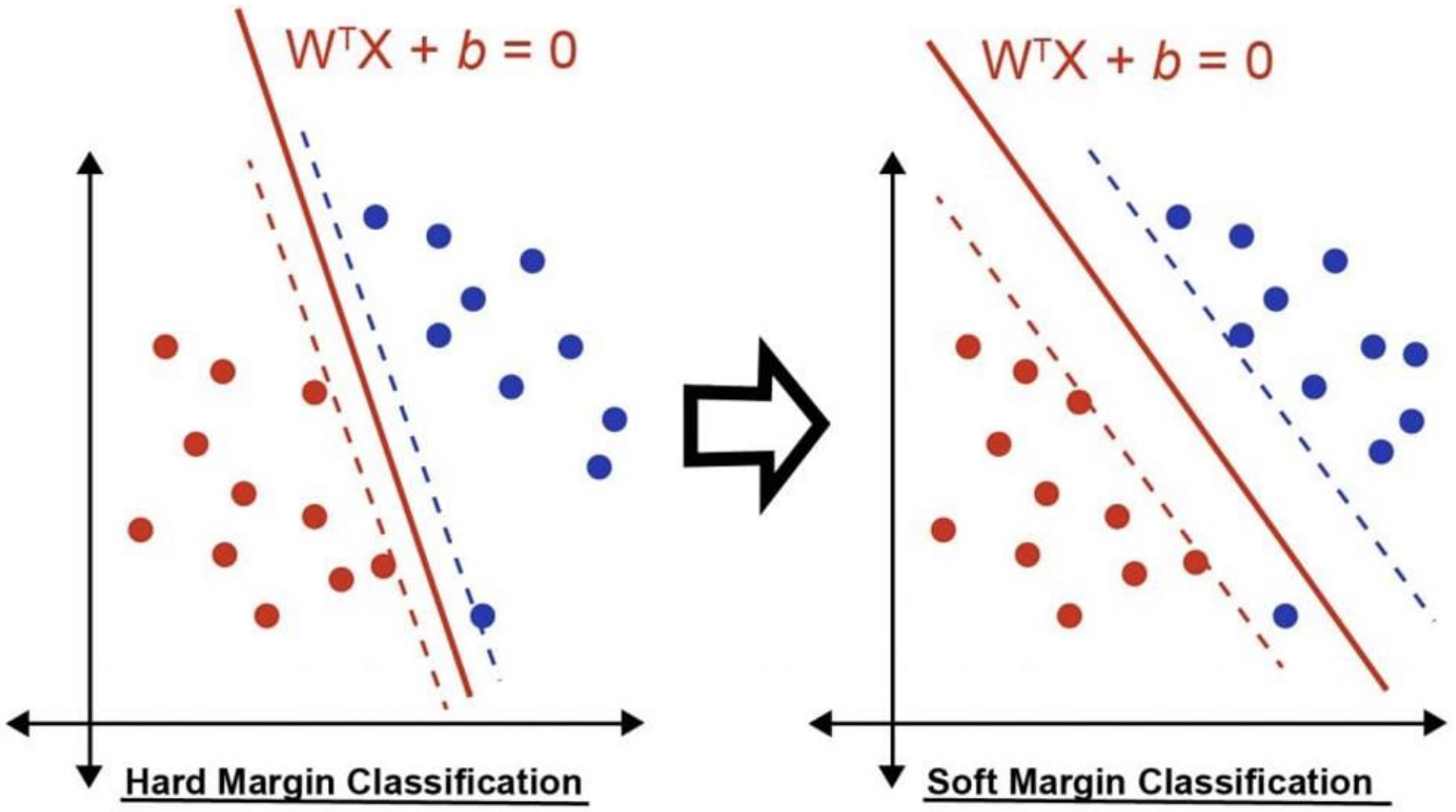
$min_{\mathbf{w},b} \frac{\mathbf{w^T w}}{2}$

subject to: $y_i(\mathbf{w}^T\mathbf{x_i} + b) \geq 1$ ($\forall$ data points $\mathbf{x_i}$).

Soft margin

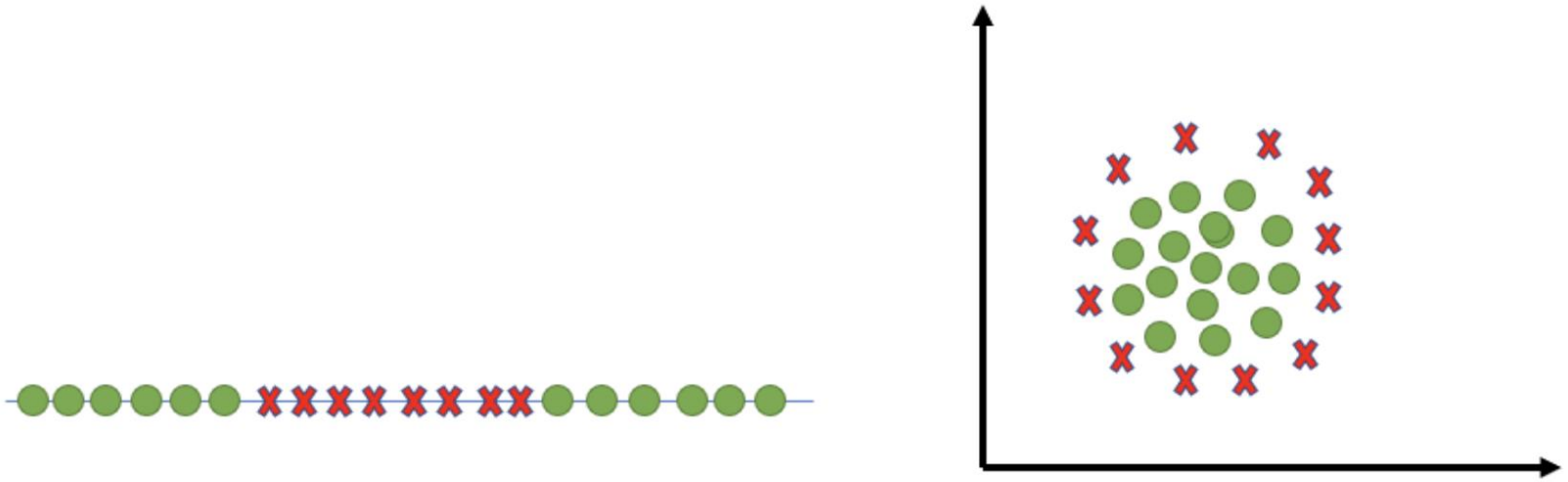
$$min_{\mathbf{w},b,\epsilon} \frac{\mathbf{w^T w}}{2} + C \sum_i \epsilon_i$$

subject to: $y_i(\mathbf{w}^T \mathbf{x_i} + b) \geq 1 - \epsilon_i$ and $\epsilon_i \geq 0$ ($\forall$ data points $\mathbf{x_i}$).

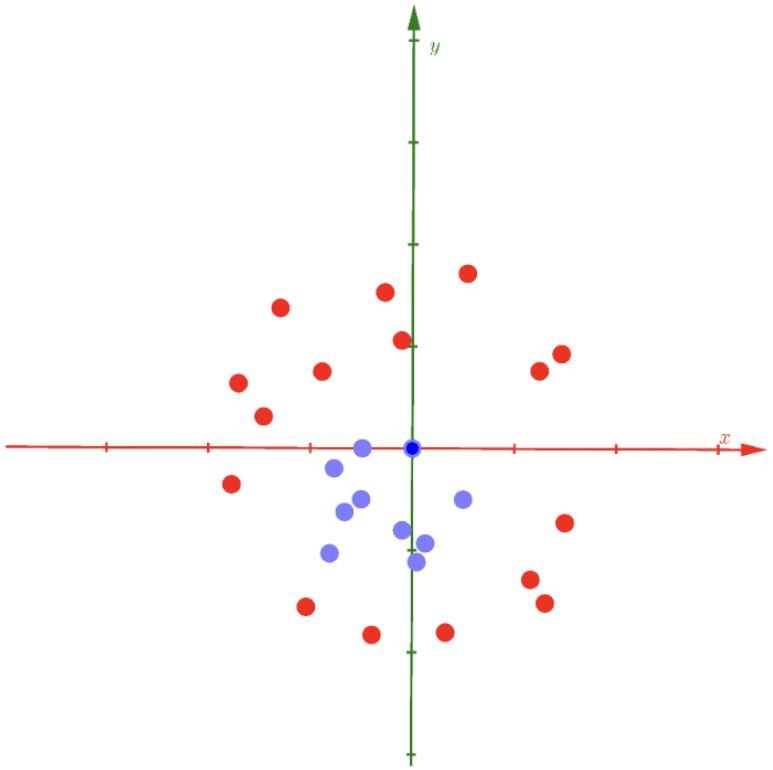


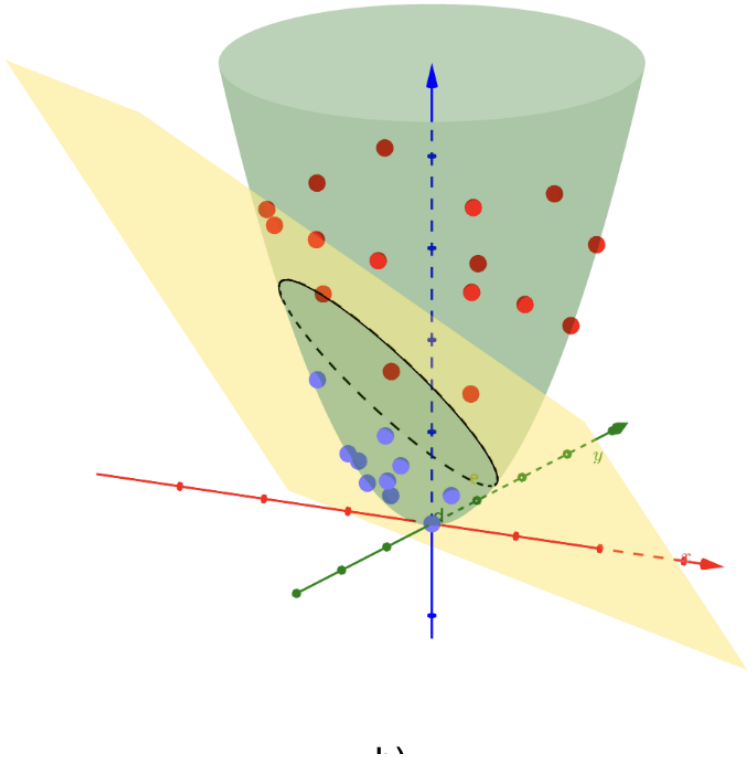
Kernels in Support Vector Machine

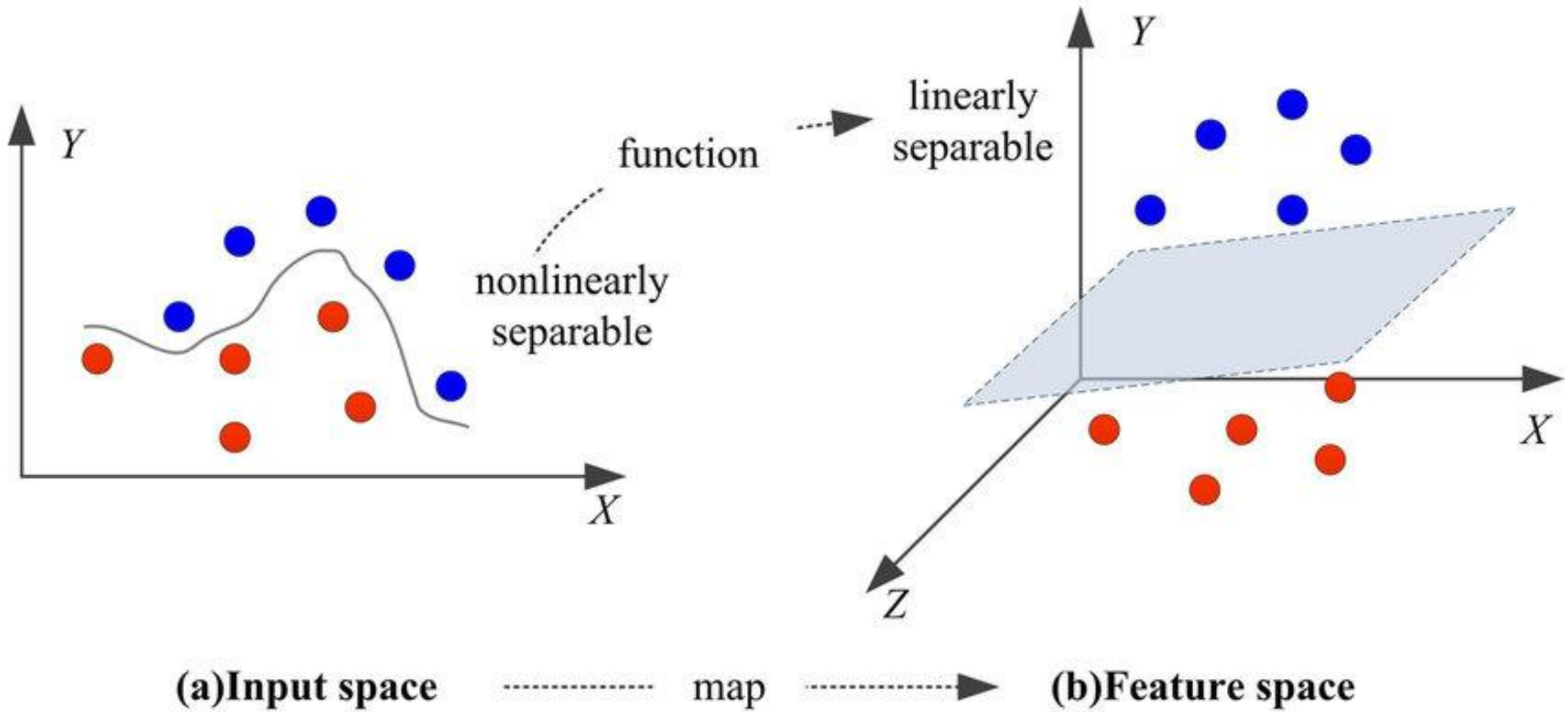
- The most interesting feature of SVM is that it can even work with a non-linear dataset and for this, we use “Kernel Trick” which makes it easier to classifies the points. Suppose we have a dataset like this:



Data Transform from the original space to higher dimension space

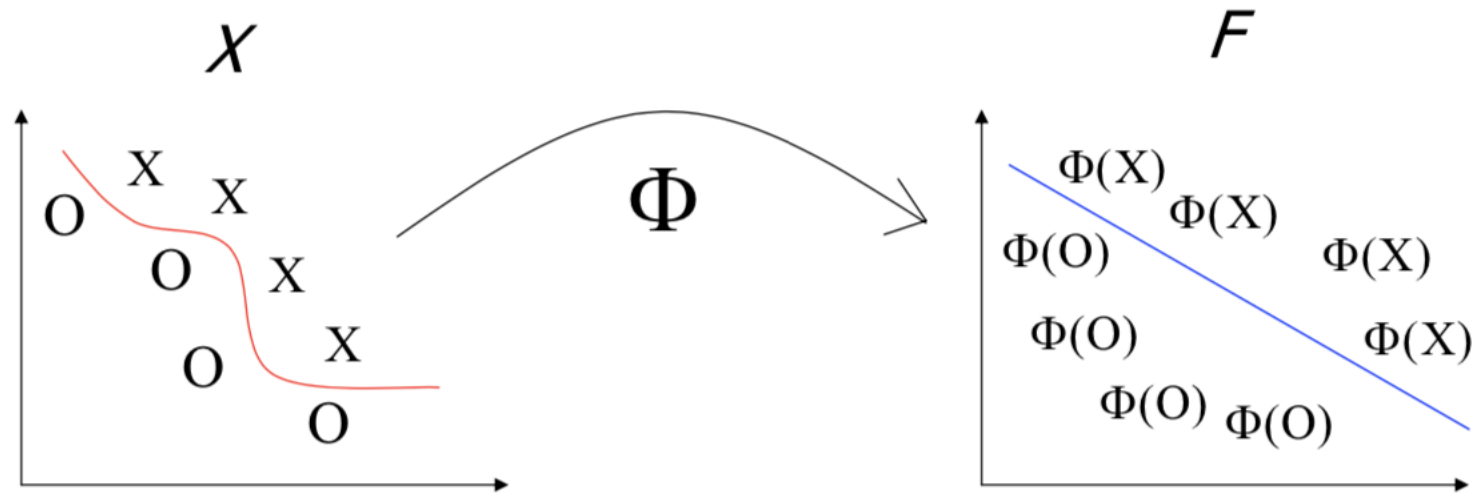





Y
X
function
nonlinearly
separable
linearly
separable
Y
X
Z
(a)Input space
map
(b)Feature space

Data Transform from the original space to higher dimension space

- Solution: Map the data into a (possibly high-dimensional) vector space where linear relations exist among the data, then apply a linear algorithm in this space



Kernel Functions

- **Role of Kernel Functions in Support Vector Machines (SVM)**
  - Kernel functions play a crucial role in SVM by enabling the algorithm to solve **non-linear classification or regression problems**. The kernel function implicitly maps input data into a higher-dimensional space where the data becomes linearly separable, even if it is not linearly separable in the original input space.

Kernel Definition

- A function that takes as its inputs vectors in the original space and returns the dot product of the vectors in the feature space is called a *kernel function*
- More formally, if we have data $\mathbf{x}, \mathbf{z} \in X$ and a map $\phi : X \rightarrow \Re^N$ then

$$k(\mathbf{x}, \mathbf{z}) = \left\langle \phi(\mathbf{x}), \phi(\mathbf{z}) \right\rangle$$

is a kernel function

Kernel definition: an important point

- Using kernels, we do not need to embed the data into the space $\Re^N$ explicitly, because a number of algorithms only require the inner products between image vectors!
- We never need the coordinates of the data in the feature space!

Kernel Example

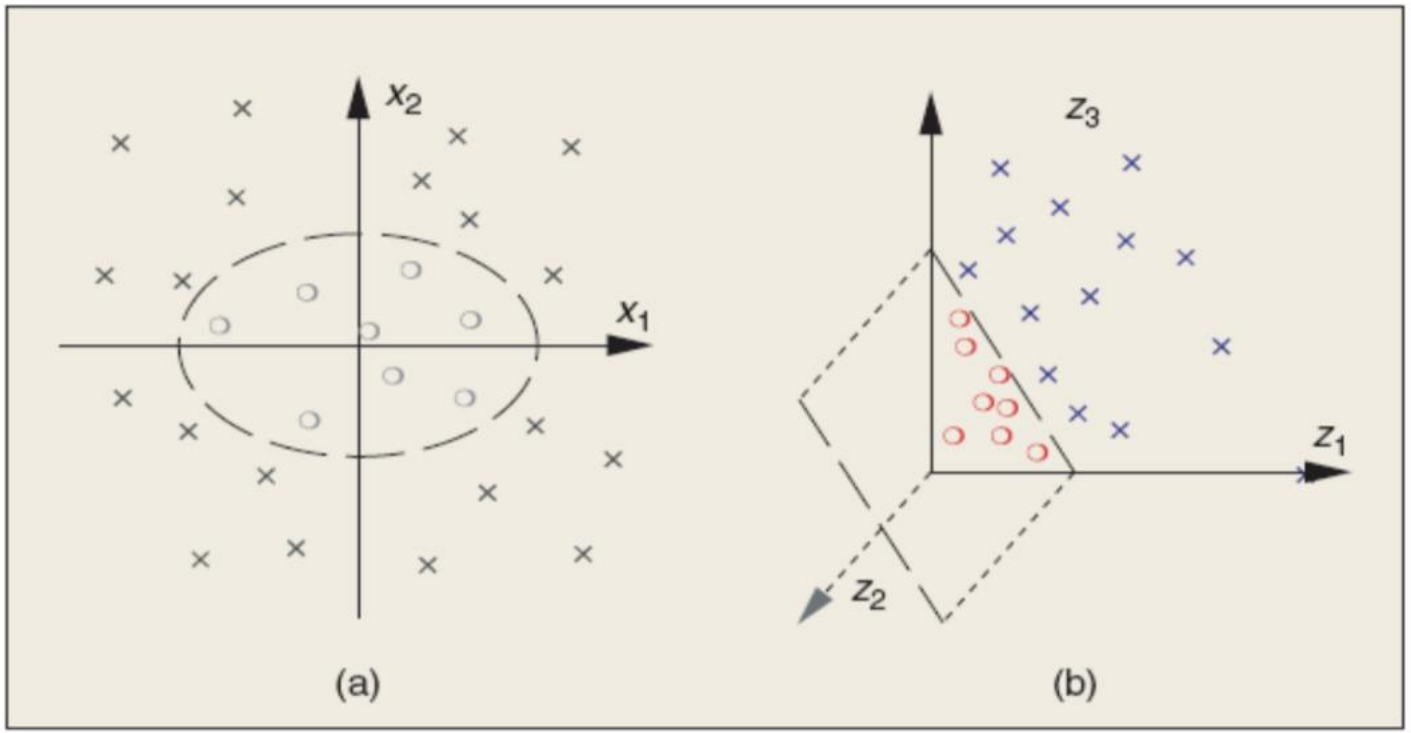
- Consider a two-dimensional input space $X \subseteq \Re^2$ with the feature map:

$$\phi : \mathbf{x} = (x_1, x_2) \mapsto \phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2) \in F = \Re^3$$

Now consider the inner product in the feature space:

$$\begin{aligned} \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle &= \langle (x_1^2, x_2^2, \sqrt{2}x_1x_2), (z_1^2, z_2^2, \sqrt{2}z_1z_2) \rangle \\ &= x_1^2z_1^2 + x_2^2z_2^2 + 2x_1x_2z_1z_2 \quad = (x_1z_1 + x_2z_2)^2 \\ &= \langle \mathbf{x}, \mathbf{z} \rangle^2 \end{aligned}$$

Kernel example



▲ 1. Effect of the map $\phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$. (a) Input space $\mathcal{X}$ and (b) feature space $\mathcal{H}$.

Kernel example

- Then $k(\mathbf{x},\mathbf{z}) = \langle \mathbf{x},\mathbf{z} \rangle^2$
- But $k(\mathbf{x},\mathbf{z})$ is also the kernel that computes the inner product of the map

$$\psi(\mathbf{x}) = (x_1^2, x_2^2, x_1 x_2, x_2 x_1) \in F = \Re^4$$



- This shows that a given feature space is not unique to a given kernel function

Kernel Method

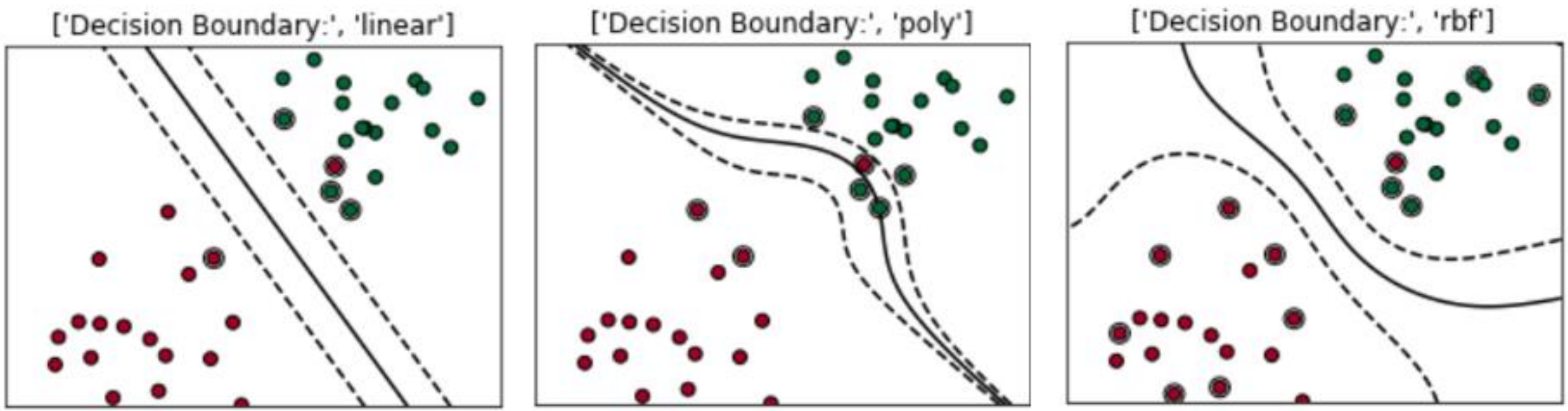
- Idea:
  - Define $K : X \times X \to \mathbb{R}$, called kernel, such that:

$$\Phi(x) \cdot \Phi(y) = K(x, y).$$

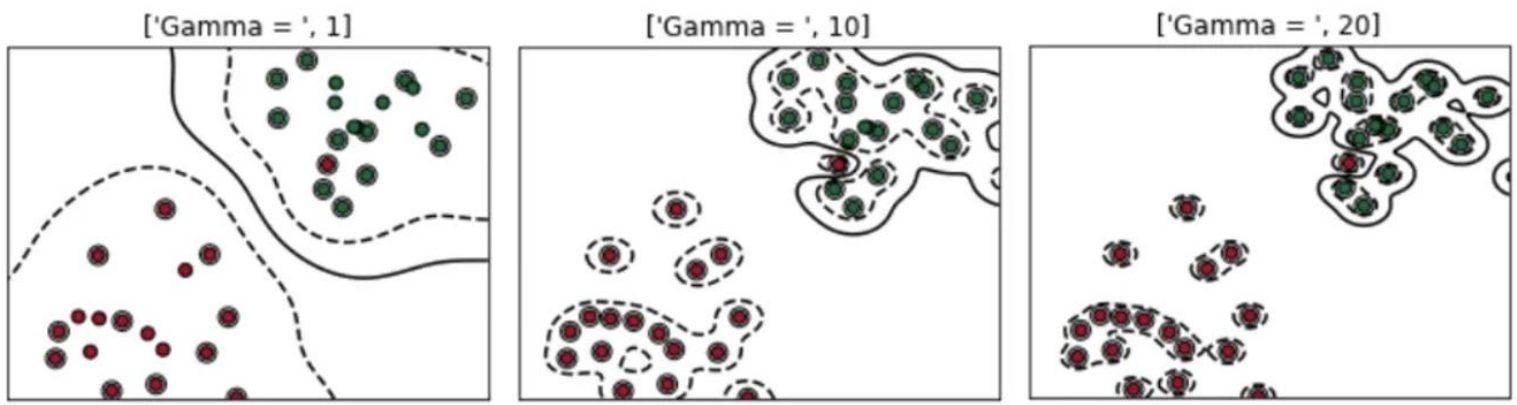
  - K often interpreted as a similarity measure.
- Benefits:
  - Efficiency: K is often more efficient to compute than Φ and the dot product.
  - Flexibility: K can be chosen arbitrarily so long as the existence of Φ is guaranteed (Mercer's condition).

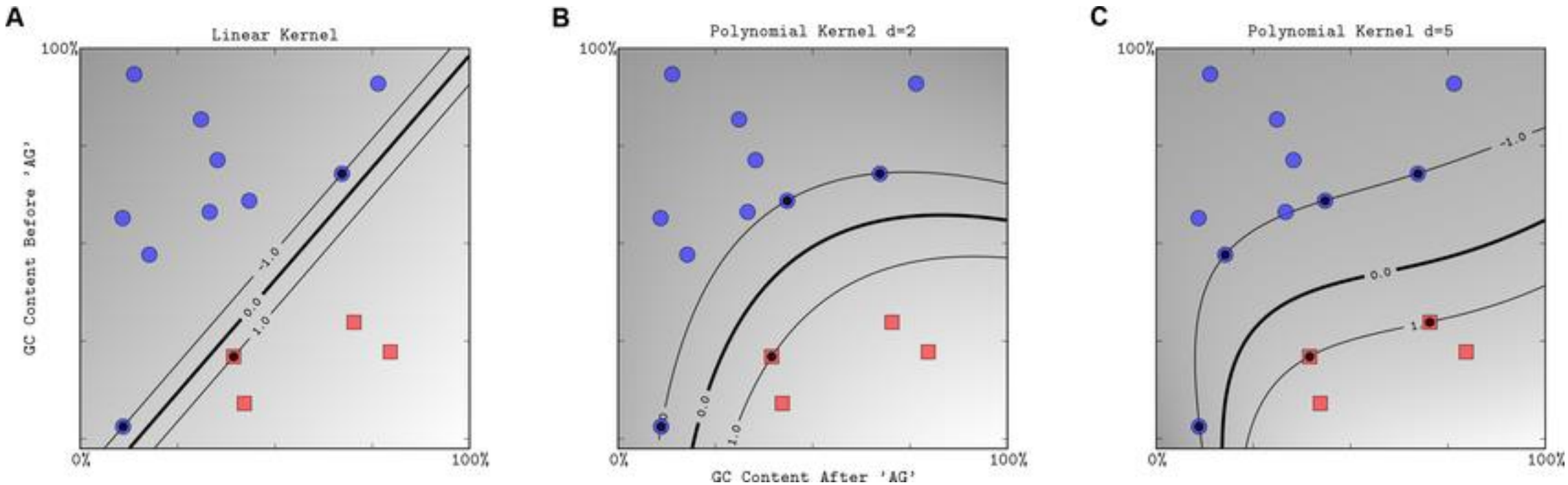
Different kernel

- **Linear Kernel:** $K(x, x') = x^T x'$. This kernel does not actually transform the data and is equivalent to the standard linear SVM.
- **Polynomial Kernel:** $K(x, x') = (\gamma x^T x' + r)^d$, where d is the degree of the polynomial, γ is a scale factor, and r is a constant term.
- **Radial Basis Function (RBF) Kernel:** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$, where γ is a scale factor. The RBF kernel is particularly popular due to its flexibility in handling non-linear relationships.
- **Sigmoid Kernel:** $K(x, x') = \tanh(\gamma x^T x' + r)$, which resembles the activation function used in neural networks.


['Decision Boundary:', 'linear']
['Decision Boundary:', 'poly']
['Decision Boundary:', 'rbf']

```
# Gamma is small, influence is small
clf = svm.SVC(kernel='rbf', Gamma=1)
# Gamma gets bigger, influence increase, the decision boundary get
wiggled
clf2 = svm.SVC(kernel='rbf', Gamma=10)
# Gamma gets too big, influence too much, the decision boundary get
too wiggled
clf3 = svm.SVC(kernel='rbf', Gamma=20)
```




A
Linear Kernel
100%
0%
100%
GC Content Before 'AG'
-1.0
0.0
1.0
B
Polynomial Kernel d=2
100%
0%
100%
GC Content After 'AG'
0.0
1.0
C
Polynomial Kernel d=5
100%
0%
100%
-1.0
0.0
1

Learning SVM

The original SVM optimization problem is given as follows:

$$\min_{w,b} \frac{1}{2}\|w\|^2$$
$$\text{subject to } y^{(i)}(w^T x^{(i)} + b) \geq 1 \text{ for all } i = 1, \cdots, M$$

Using the concept of ***Duality***, the original SVM can be reformulated into its dual form, which is as follows.

$$\max_{\alpha} \sum_{i=1}^{M} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{M} y^{(i)} y^{(j)} \alpha_i \alpha_j x^{(i)} . x^{(j)}$$
$$\text{subject to } 0 \leq \alpha_i \leq C \text{ for all } i, \text{ and } \sum_{i=1}^{M} \alpha_i y^{(i)} = 0.$$

where α_i are the Lagrange multipliers, C is the penalty parameter for the soft margin. For more details on the concept of duality and why the above is the dual formulation of SVMs, see this blog article on ***Duality***.

Learning SVM with Kernel function

The above dual expression, can be easily extended if we replace $x^{(i)}.x^{(j)}$ with a more general kernel function: $K(x^{(i)}, x^{(j)})$. The formulation is:

$$\max_{\alpha} \sum_{i=1}^{M} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{M} y^{(i)} y^{(j)} \alpha_i \alpha_j K(x^{(i)}, x^{(j)})$$

$$\text{subject to } 0 \leq \alpha_i \leq C \text{ for all } i, \text{ and } \sum_{i=1}^{M} \alpha_i y^{(i)} = 0.$$

This dual formulation allows the SVM to incorporate the kernel trick, enabling the algorithm to learn non-linear decision boundaries by implicitly mapping inputs into a high-dimensional space.

Inference in SVM

The test prediction for a Kernel SVM is given by:

$$y_{\text{pred}} = \text{sign}\left(\sum_{i=1}^{M} \alpha_i y^{(i)} K(x^{(i)}, x_{\text{test}}) + b\right)$$

where:

- x_{test} is a new test example for which we want to predict the label.
- $x^{(i)}$ are the support vectors, which are the training examples that have non-zero α_i.
- $y^{(i)}$ are the labels of the support vectors.
- $K(x^{(i)}, x_{\text{test}})$ is the kernel function applied between the support vector $x^{(i)}$ and the test example x_{test}.
- b is the bias term, which is determined using complementary slackness ***Duality Blog***. To recap, we take the points i which are support vectors: in that case, $\alpha_i > 0$. Using complementary slackness, $b = y^{(i)} - w^T x^{(i)} = b = y^{(i)} - \sum_{j=1}^{M} \alpha_i y^{(i)} K(x^{(i)}, x^{(j)})$. To obtain a more robust estimate of b, it is common practice to average b over all support vectors on the margin:

$$b = \frac{1}{|S|} \sum_{i \in S} \left(y^{(i)} - \sum_{j=1}^{M} \alpha_j y^{(j)} K(x^{(i)}, x^{(j)})\right)$$

where S denotes the set of indices of support vectors on the margin.

Dual optimization problem

- **Constrained optimization:**

$$\max_{\alpha} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$$

$$\text{subject to: } \alpha_i \geq 0 \wedge \sum_{i=1}^{m} \alpha_i y_i = 0, i \in [1, m].$$

- **Solution:**

$$h(x) = \text{sgn}\Big(\sum_{i=1}^{m} \alpha_i y_i (\mathbf{x}_i \cdot \mathbf{x}) + b \Big),$$

$$\text{with } \; b = y_i - \sum_{j=1}^{m} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_i) \; \text{ for any SV } \mathbf{x}_i \, .$$

SVM with Kernel

- **Constrained optimization:**

$$\max_{\alpha} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

$$\text{subject to: } 0 \leq \alpha_i \leq C \wedge \sum_{i=1}^{m} \alpha_i y_i = 0, i \in [1, m].$$

- **Solution:**

$$h(x) = \text{sgn}\Big(\sum_{i=1}^{m} \alpha_i y_i K(x_i, x) + b\Big),$$

with $b = y_i - \sum_{j=1}^{m} \alpha_j y_j K(x_j, x_i)$ **for any** x_i **with** $0 < \alpha_i < C.$

SMO: Optimization

- Problem

$$\min_{\alpha_i} F(\boldsymbol{\alpha}) = \frac{1}{2}\sum_{i,j=1}^{l} y_i y_j \alpha_i \alpha_j K_{ij} - \sum_{i=1}^{l} \alpha_i$$

$$\text{s.t.: } 0 \le \alpha_i \le C, i = 1,\ldots,l,$$

$$\sum_{k=1}^{l} \alpha_k y_k = 0$$

$$\rightarrow \forall (i,j) : y_i \alpha_i + y_j \alpha_j = const$$

$$\rightarrow \alpha_j = y_j (const - y_i \alpha_i)$$

- Fixing all $\alpha_k, k \neq i, j$

$$F(\boldsymbol{\alpha}) = F(\alpha_i) = A\alpha_i^2 + B\alpha_i + C$$

- Updating scheme

(without the box constraint)

$$\alpha_i^{new} = \alpha_i^{old} + \frac{y_i\left(E_j^{old} - E_i^{old}\right)}{2\kappa_{ij}},$$

$$\alpha_j^{new} = \alpha_j^{old} + \frac{y_j\left(E_i^{old} - E_j^{old}\right)}{2\kappa_{ij}}.$$

$$E_i = \sum_{k=1}^{l} y_k \alpha_k K(x_k, x_i) - y_i, i = 1,\ldots,l,$$

$$\kappa_{ij} = K_{ii} + K_{jj} - 2K_{ij}$$

Summary of SVM

- 1
- 2
- 3
- 4
- 5